

Maîtriser l'environnement **Scala/SBT avec Visual Studio Code** et créer un dépôt sur **GitLab**.

Rôle de SBT (Simple Build Tool)

- **Compiler le code Scala** → transforme tes fichiers .scala en bytecode Java.
- **Gérer les dépendances** → télécharge automatiquement les bibliothèques depuis Maven Central ou autres dépôts.
- **Structurer le projet** → impose une organisation standard (src/main/scala, src/test/scala, etc.).
- **Exécuter et tester** → permet de lancer ton application (sbt run) et tes tests (sbt test).
- **Packager** → génère des fichiers .jar pour déployer ton application.
- **Automatiser** → s'intègre dans des pipelines CI/CD (GitLab CI, Jenkins, GitHub Actions).

Pourquoi utiliser SBT

- C'est l'équivalent de **Maven/Gradle** pour Java, mais adapté à Scala.
- Il simplifie la gestion des projets complexes.
- Il est reconnu comme la norme dans l'écosystème Scala.

1. Installer l'environnement Scala + SBT

- **Installer Java JDK** : Scala nécessite un JDK (version 17 recommandée).
- **Installer SBT** : SBT est l'outil de build officiel pour Scala. Téléchargez-le depuis scala-sbt.org.
- Vérifiez l'installation :

```
java -version
```

```
sbt sbtVersion
```

2. Configurer Visual Studio Code

- Installez **VS Code**.
- Ajoutez l'extension **Metals (Scala Language Server)** : elle fournit autocomplétion, navigation, refactoring et exécution.
- Ajoutez éventuellement **Scala Syntax (official)** pour la coloration syntaxique.

- Ouvrez votre projet Scala dans VS Code, Metals détectera automatiquement le fichier build.sbt.

3. Créer un projet Scala avec SBT

Dans un terminal :

```
sbt new scala/scala-seed.g8
```

- Donnez un nom au projet (ex. moteur-recommandation).
- Cela génère une structure :

Code

```
moteur-recommandation/
```

```
├─ build.sbt
```

```
├─ project/
```

```
└─ src/
```

```
    ├─ main/scala/
```

```
    └─ test/scala/
```

- Lancez le projet :

```
cd moteur-recommandation
```

```
sbt run
```

4. Créer un dépôt GitLab

1. **Créer un compte GitLab** si ce n'est pas déjà fait.
2. Dans GitLab, créez un **nouveau projet** → choisissez **vide**.
3. Copiez l'URL du dépôt (ex. <https://gitlab.com/username/moteur-recommandation.git>).
4. Dans votre projet local :

```
git init
```

```
git remote add origin https://gitlab.com/username/moteur-recommandation.git
```

(Vérification de versionning) :

```
C:\Users\username\moteur-recommandation>git remote -v
```

```
git add .
```

```
git commit -m "Initial commit"
```

```
git push -u origin master
```

5. Workflow de développement

- **Développement** : codez dans src/main/scala.
- **Tests unitaires** : placez-les dans src/test/scala.
- **Build & run** : utilisez sbt compile, sbt run, sbt test.
- **Commit régulier** :

bash

git add .

git commit -m "Ajout fonction similarité Jaccard"

git push

- **Collaboration** : chaque membre clone le dépôt GitLab et travaille sur des branches (git checkout -b feature-similarity).

6. Bonnes pratiques

- Toujours écrire un **README.md** expliquant comment lancer le projet.
- Utiliser **Metals** pour exécuter directement les fichiers Scala dans VS Code.
- Documenter les fonctions dans le code avec des commentaires clairs.
- Faire des commits fréquents et explicites.

En résumé :

1. Installez JDK + SBT.
2. Configurez VS Code avec Metals.
3. Créez un projet Scala via sbt new.
4. Poussez le projet sur GitLab.
5. Travaillez en équipe avec branches et commits réguliers.

7. Les fondamentaux de SBT :

Pour lancer ton projet Scala avec **SBT** dans **Visual Studio Code**, tu dois installer **l'extension Metals**, ouvrir ton projet contenant build.sbt, puis utiliser les commandes intégrées (sbt run, sbt compile, etc.) directement depuis le terminal ou via Metals.

Préparer l'environnement

- **Installer Java JDK** : minimum JDK 11 (souvent JDK 17 recommandé).
- **Installer SBT** : télécharge depuis sbt download (scala-sbt.org in Bing).
- **Installer VS Code** : ajoute l'extension **Metals (Scala Language Server)** depuis le Marketplace.

Structure d'un projet SBT

Ton projet doit contenir :

- build.sbt → configuration du projet (version Scala, dépendances).
- project/build.properties → version de SBT.
- src/main/scala/ → ton code source.
- src/test/scala/ → tes tests unitaires.

Metals détecte automatiquement cette structure et active l'environnement Scala.

Lancer le projet dans VS Code

1. Ouvre ton dossier projet dans VS Code.
2. Metals va reconnaître build.sbt et proposer d'**importer le projet**.
3. Une fois importé, tu peux :
 - Compiler :

sbt compile

- Exécuter :

sbt run

- Tester :

sbt test

4. Tu peux aussi utiliser la **palette de commandes VS Code** (Ctrl+Shift+P) → taper Metals: Run ou Metals: Test.

Exemple minimal Hello World

- build.sbt :

```
scalaVersion := "3.3.1"
```

- src/main/scala/Hello.scala :

```
@main def helloWorld = println("Hello, world")
```

- Lancer avec :

sbt run

Points de vigilance

- **JAVA_HOME** doit pointer vers ton JDK.
- Si Metals ne détecte pas ton projet, vérifie que build.sbt est bien à la racine.
- Mets à jour SBT et Metals pour éviter les incompatibilités.

Résumé

- Installe **JDK + SBT + Metals**.
- Ouvre ton projet dans VS Code → Metals importe automatiquement.
- Utilise sbt run pour lancer ton code.
- Vérifie la structure standard (build.sbt, src/main/scala).

Les commandes de base :

Quand tu lances la commande :

sbt compile

Voici ce que tu dois obtenir et comprendre :

Ce que fait sbt compile

- SBT lit ton fichier **build.sbt** et télécharge les dépendances nécessaires (depuis Maven Central ou autres dépôts).
- Il compile ton code Scala (dans `src/main/scala`) en **bytecode Java** (.class) placé dans le dossier **target/scala-<version>/classes**.
- Il compile aussi tes tests (dans **src/test/scala**) si tu en as.

Résultat attendu

- Si tout est correct, tu verras un message du type :

[success] Total time: 3 s, completed Jul 28, 2026, 13:05:12

- Cela signifie que la compilation s'est bien déroulée.
- Tu trouveras les fichiers compilés dans le dossier target.

En cas d'erreurs

- Si ton code contient une erreur de syntaxe ou de type, SBT affichera un message d'erreur détaillé (ligne concernée, type attendu, etc.).
- Exemple :

[error] src/main/scala/Hello.scala:5: not found: value println

[error] println("Hello, world")

Tu devras corriger ton code avant de relancer sbt compile.

Étape suivante après compilation

Une fois compilé avec succès, tu peux exécuter ton projet avec :

sbt run

Cela lancera ton application Scala en utilisant les classes compilées.

Résumé

- sbt compile → compile ton code et vérifie qu'il est correct.
- Résultat attendu : [success] + fichiers .class dans target.
- En cas d'erreurs, SBT te donne les détails pour corriger.
- Après compilation réussie, tu peux lancer ton projet avec sbt run.